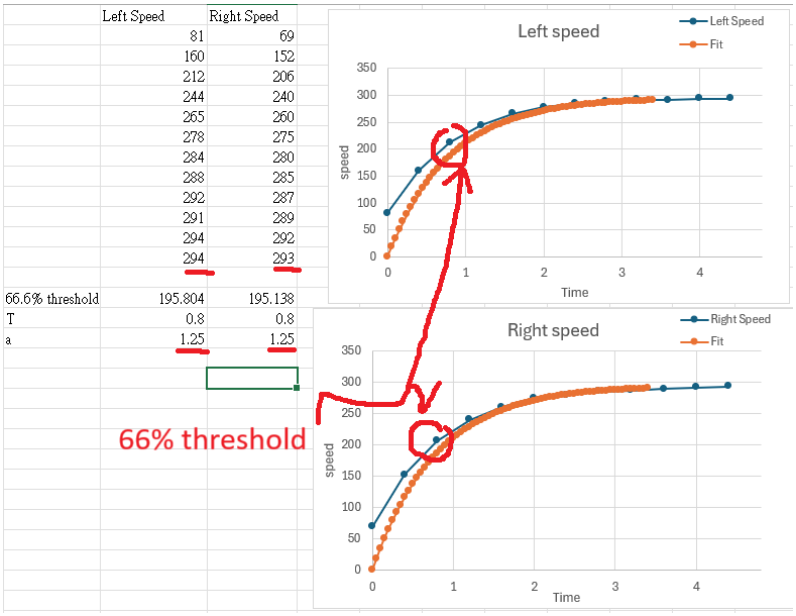
EIE3123 Dynamic Electronic System && EIE3128 IoT workshop and Project

Joint Lab

Name: Wong Chun Hei

Design of Control system:

1st design of compensated system:



Left wheel	Right wheel
• Final value=294 • 294*66%=195 • When y=195, x=0.8(as the graph is in ms unit) • $\alpha = \frac{1}{\tau} = \frac{1}{0.8} = 1.25$	• Final value=293 • 293*66%=195 • When y=195, x=0.8(as the graph is in ms unit • $\alpha = \frac{1}{\tau} = \frac{1}{0.8} = 1.25$

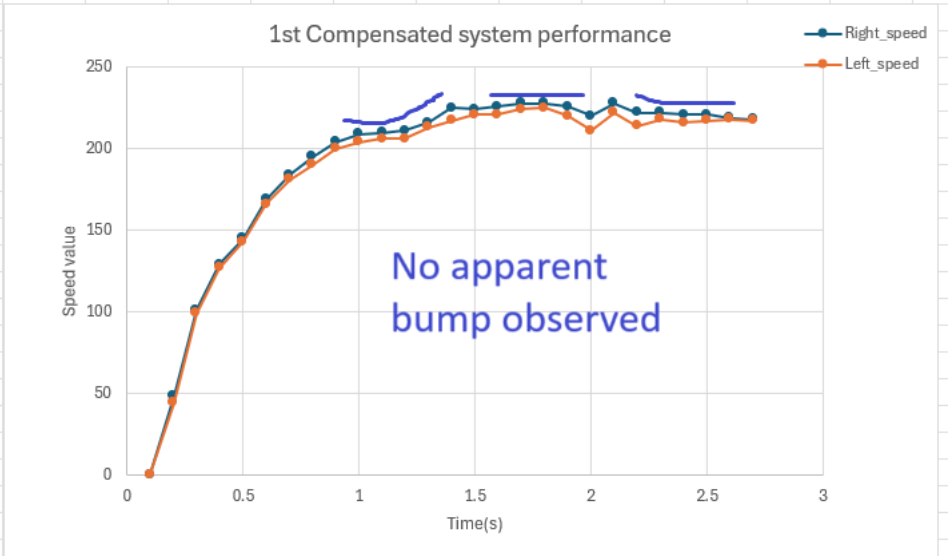
Left and right speed parameters from data collecting through USART

Using the above information(robot car max PWM value = 65535),

left $G_s = 294 * \frac{(1.25)}{s+1.25} * (\frac{1}{65535})$

right $G_s = 293 * \frac{*(1.25)}{s+1.25} * (\frac{1}{65535})$

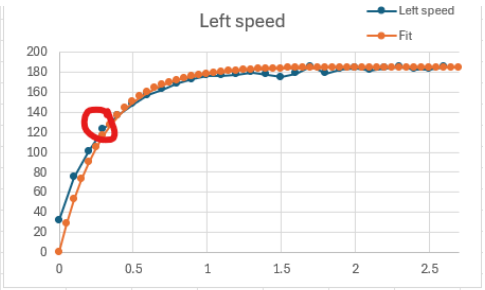
Performance optimization(skipping the 1st design details because the result is unsatisfactory):

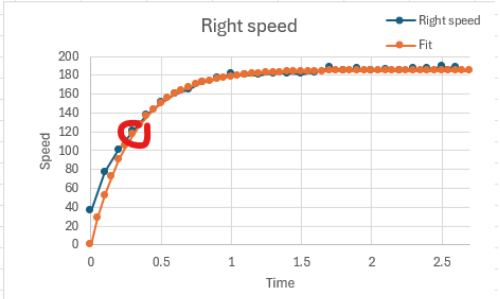


After completing the 1st system design and implementation, the car can move roughly a straight line toward B, yet the rate for the speed difference controller isn't fast enough to get the desired PWM. A quicker response time can be achieved by increasing Kp. Moreover, the graph plotted based on the actual run isn't perfect enough to show the 3 'pumps', which shows the controller cannot successfully chase back the desired PWM in quick response time. Therefore, I tried increasing the sampling rate per second for more accurate measurements of a and T.

2nd design of the compensated system:

	Left speed	Right speed
	32	36
	75	77
	101	101
	123	121
	137	138
	148	151
	157	160
	163	165
	169	173
	173	177
	177	181
	177	180
	178	180
	180	182
	178	182
	175	181
	179	183
	186	188
	179	185
	183	187
	184	185
	182	186
	184	185
	185	187
	183	187
	183	189
	185	188
66% threshold	123.21	125.208
T	0.3	0.3
a	3.333333333	3.333333333





Left wheel
• Final value=185 • 185*66%=123 • When y=123, x=0.3(as the graph is in ms unit) • $\alpha = \frac{1}{\tau} = \frac{1}{0.3} = 3.33$
Right wheel
• Final value=188 • 188*66%=125 • When y=125, x=0.3(as the graph is in ms unit $\alpha = \frac{1}{\tau} = \frac{1}{0.3} = 3.33$

left $G_s = 185 * \frac{(3.33)}{s+3.33} * (\frac{1}{65535})$

Left wheels (desired freq:9.96rad/s,

magnitude = -61dB)

$$k_p = 20\log_{10} k_p = 61dB = 1122$$

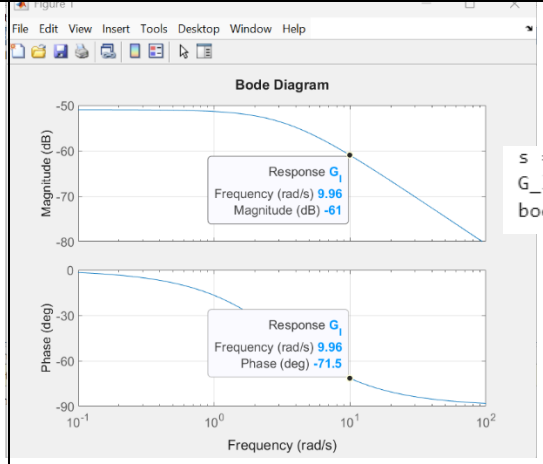
$$\frac{ki}{kp} = 9.96/10$$

$$k_i = 1117.512$$

Left $G_{cs} = 1122 * (s + 0.996)/s$

Z-transformed: $G_{clz} = (1178\,z - 1066)/(z - 1)$

Discrete digital format: $D[k] = D[k - 1] + 1178e[k] - 1066e[k - 1]$



Bode plot for Kp and Ki

```
s = tf('s');
G_l = 185*3.33/(s+3.33)/65535;
bode(G_l);grid on;
```

z-transform and step response for left Gcs

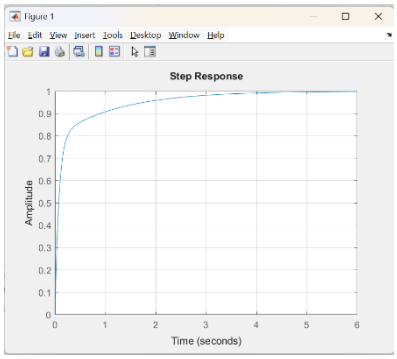
(0.1s sampling time)

```
Gc_l = 1122*(s+0.996)/s;
c2d(Gc_l, 0.1, 'tustin')
step(feedback(G_l*Gc_l, 1)); grid on;
```

```
ans =

  1178 z - 1066
  -------------
       z - 1

Sample time: 0.1 seconds
Discrete-time transfer function.
```



right $G_s = 188 * \frac{*(3.33)}{s+3.33} * (\frac{1}{65535})$

Right wheels (desired freq: 9.96rad/s,

magnitude = -60.8dB)

$$k_p = 20\log_{10} k_p = 60.8dB = 1096$$

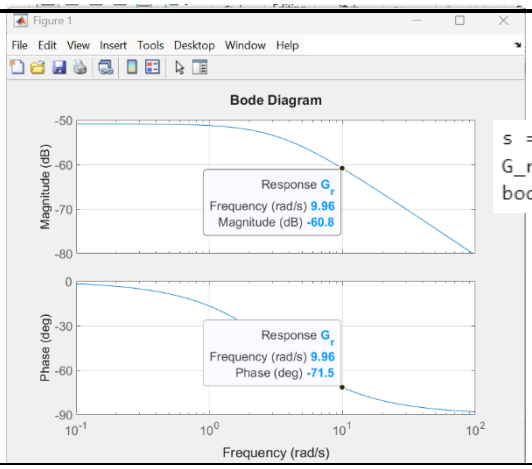
$$\frac{ki}{kp} = 9.96/10$$

$$k_i = 1091.616$$

Right $G_{cs} = 1096 * (s + 0.996)/s$

Z-transformed: $G_{clz} = (1151\,z - 1041)/(z - 1)$

Discrete digital format: $D[k] = D[k - 1] + 1151e[k] - 1041e[k - 1]$



Bode plot for Kp and Ki

```
s = tf('s');
G_r = 188*3.33/(s+3.33)/65535;
bode(G_r);grid on;
```

z-transform and step response for right Gcs

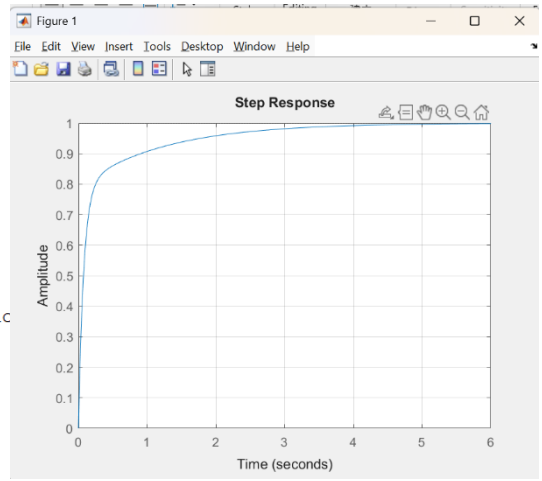
(0.1s sampling time)

```
Gc_r = 1096*(s+0.996)/s;
c2d(Gc_r, 0.1, 'tustin')
step(feedback(G_r*Gc_r, 1)); grid on;
```

```
ans =

  1151 z - 1041
  -------------
       z - 1

Sample time: 0.1 seconds
Discrete-time transfer func
```



Speed difference controller (close-loop transfer function of individual wheel)

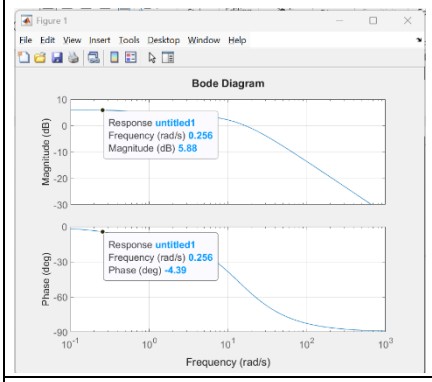
Speed difference controller (desired freq: 0.256rad/s, magnitude = +5.88dB)

$$k_p = 20\log_{10} k_p = -5.88dB = 0.5081$$

$$\frac{ki}{kp} = 0.256/10$$

$$k_i = 0.013$$

Outer Loop PI controller $G_{cs} = 0.5081 * (s + 0.0256)/s$

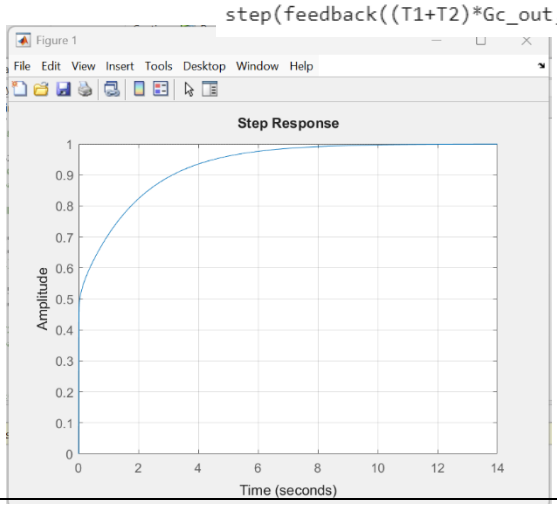


Bode plot for Kp and Ki

```
T1 = feedback(G_l*Gc_l, 1);
T2 = feedback(G_r*Gc_r, 1);
bode(T1+T2);grid on;
```

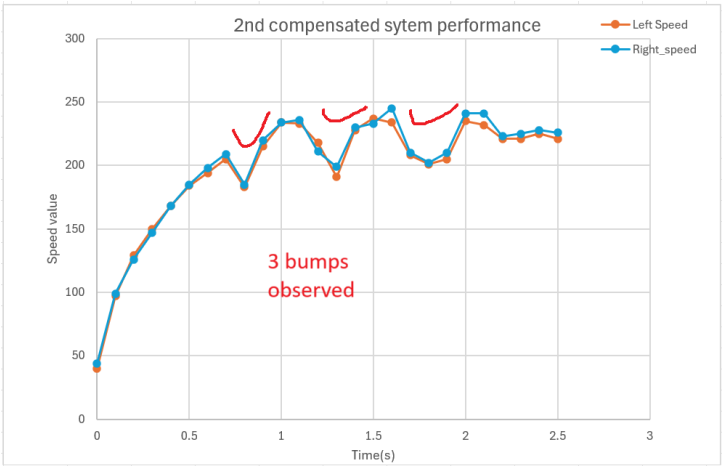
step response for speed difference controller

```
Gc_out = 0.5081*(s+0.0256)/s;
step(feedback((T1+T2)*Gc_out, 1)); grid on;
```



Results:

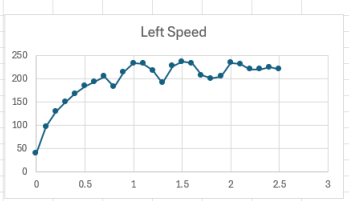
This time the 3rd bump can be seen. This might be caused by the Ki increase for the left and right Pi controllers, providing better steady-state performance.

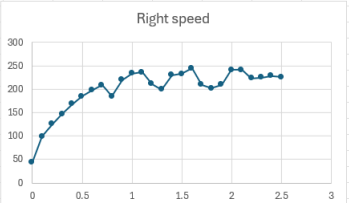


Also, both wheels of the robot car reach similar counter values(5110, 5174) at the end of the run. I think the system performs well according to the requirements, but it still lacks the stability. As shown in the video, the robot car can run straight but sometimes it differ a little from the straight line orientation on the race map.

2nd try compensated system

Time	Left Counter	Right Counter		Left Speed	Right speed
0	1	-1		40	44
0.1	41	43		97	99
0.2	138	142		129	126
0.3	267	268		150	147
0.4	417	415		168	168
0.5	585	583		184	185
0.6	769	768		194	198
0.7	963	966		205	209
0.8	1168	1175		183	185
0.9	1351	1360		215	220
1	1566	1580		234	234
1.1	1800	1814		233	236
1.2	2033	2050		218	211
1.3	2251	2261		191	199
1.4	2442	2460		228	230
1.5	2670	2690		237	233
1.6	2907	2923		234	245
1.7	3141	3168		208	210
1.8	3349	3378		201	202
1.9	3550	3580		205	210
2	3755	3790		235	241
2.1	3990	4031		232	241
2.2	4222	4272		221	223
2.3	4443	4495		221	225
2.4	4664	4720		225	228
2.5	4889	4948		221	226
2.6	5110	5174			





Implementation of control system(mainly Pi controller functions):

01 Variable setting

```
// PI controller parameters for Motor 1 Left
float Kp1 = 1122; // Proportional gain for Motor 1
float Ki1 = 1117.512; // Integral gain for Motor 1

// PI controller parameters for Motor 2 Right
float Kp2 = 1096; // Proportional gain for Motor 2
float Ki2 = 1091.616; // Integral gain for Motor 2


// PI controller parameters for Speed Synchronization
float Kp_sync = 0.5081; // Proportional gain for synchronization
float Ki_sync = 0.013; // Integral gain for synchronization

// Control variables for Motor 1
float previous_error1 = 0.0;
float integral1 = 0.0;
float current_duty_cycle1 = 0.0;
float previous_duty_cycle1 = 0.0;

// Control variables for Motor 2
float previous_error2 = 0.0;
float integral2 = 0.0;
float current_duty_cycle2 = 0.0;
float previous_duty_cycle2 = 0.0;

// Control variables for synchronization
float previous_sync_error = 0.0;
float sync_integral = 0.0;

// Desired speeds
float desired_speed1 = 52450.0; // Target speed for Motor 1 Left
float desired_speed2 = 53600.0; // Target speed for Motor 2 Right
//52450.0,53600.0

float left_wheel_counter;
float right_wheel_counter;
```

Gain stored in variable, used for 3 PI controller

Variable instances for PI controller

Desired PWM value to run straight line for my robot car

02 Functions(Left motor controller similar to right motor):

02 Functions (PI controller)

```
// Function to control Motor 1 using PI controller
void pi_control_motor1(float current_speed) {
    float error = desired_speed1 - current_speed;
    integral1 += error;

    if (integral1 > 65535) integral1 = 65535;
    if (integral1 < 0) integral1 = 0;

    current_duty_cycle1 = previous_duty_cycle1 + Kp1 * (error - previous_error1) + Ki1 * integral1;
    current_duty_cycle1 = (current_duty_cycle1 < 0) ? 0 : (current_duty_cycle1 > 65535) ? 65535 : current_duty_cycle1;
    motor((int32_t)current_duty_cycle1, right_wheel_counter);
    previous_error1 = error;
    previous_duty_cycle1 = current_duty_cycle1;
}

// Function to synchronize speeds of both motors
void synchronize_motors(float speed1, float speed2) {
    float sync_error = speed1 - speed2;
    sync_integral += sync_error;

    // Implement the synchronization control logic
    float sync_adjustment = Kp_sync * sync_error + Ki_sync * sync_integral;

    // Adjust the duty cycles based on synchronization adjustment
    current_duty_cycle1 += sync_adjustment;
    current_duty_cycle2 -= sync_adjustment;

    // Limit duty cycles
    current_duty_cycle1 = (current_duty_cycle1 < 0) ? 0 : (current_duty_cycle1 > 65535) ? 65535 : current_duty_cycle1;
    current_duty_cycle2 = (current_duty_cycle2 < 0) ? 0 : (current_duty_cycle2 > 65535) ? 65535 : current_duty_cycle2;

    // Update motors with new duty cycles
    motor((int32_t)current_duty_cycle1, (int32_t)current_duty_cycle2);
}
```

The error between current PWM value and desired PWM value

Storing accumulated error values in 'integral?' variable--> robot car will know how much PWM values are required to increase/decrease in order to get desired PWM value.

Current duty cycle | Previous duty cycle | current error | Previous error

Output respective PWM value according to current speed(in this case, the PI controller 1 is left wheel speed regulator, therefore, left pwm is adjusted.

Speed difference PI controller

Transfer functions parameters derived in Dr Loo's Video
[Left PI: -ve* T(s), Right PI: +ve*T(s)]

Output respective PWM value when slowed down

03 main loop

03 Main loop

```
if(pulse_width>=20 && __HAL_TIM_GET_COUNTER(&htim2) < 6000){
        right_wheel_counter = __HAL_TIM_GET_COUNTER(&htim4);//Right
        left_wheel_counter = __HAL_TIM_GET_COUNTER(&htim12);//Left

        float current_speed1 = (float)left_wheel_counter;
        pi_control_motor1(current_speed1);//left

        float current_speed2 = (float)right_wheel_counter;
        pi_control_motor2(current_speed2);//right

        synchronize_motors(current_speed1, current_speed2);
        time_count = time_count + 0.01;
        snprintf(msg_buffer, 5, "%f",time_count);
        HAL_UART_Transmit(&huart2, msg_buffer, 5, HAL_MAX_DELAY);
        snprintf(msg_buffer, 9, ", %d,", __HAL_TIM_GET_COUNTER(&htim2));
        HAL_UART_Transmit(&huart2, msg_buffer, 9, HAL_MAX_DELAY);
        snprintf(msg_buffer, 9, "%d \n", __HAL_TIM_GET_COUNTER(&htim5));
        HAL_UART_Transmit(&huart2, msg_buffer, 9, HAL_MAX_DELAY);
        HAL_Delay(10);
}else {
    motor(0,0);
}
```

Get respective PWM values from counter 4 and 12

Left PI speed regulator controller

Right PI speed regulator controller

Speed difference PI controller

UART sending data for final run graph plotting

Sampling rate used: 0.01s